Module 5 Week A — Reading: Regression & Evaluation Foundations

Everything in this reading is fair game for readiness checks, drills, and labs. A learner who reads this carefully should be able to complete the Week A drill and lab without attending the concept session.

What success looks like: After this reading, you can explain why models must generalize to unseen data, split a dataset into train/validation/test sets, evaluate classification and regression models with standard metrics, build linear and logistic regression pipelines in scikit-learn, apply regularization, and describe the bias-variance tradeoff.

In This Reading

1. From Description to Prediction
2. The Machine Learning (ML) Workflow
3. Data Splitting: Train, Validation, and Test Sets
4. Cross-Validation
5. Evaluation Metrics for Classification
6. Evaluation Metrics for Regression
7. Linear Regression
8. Logistic Regression
9. Regularization
10. The Bias-Variance Tradeoff
11. scikit-learn API Fundamentals

- Key Terms
- Summary
- What's Next

---

1. From Description to Prediction

In Modules 1 through 4, every task you completed was **descriptive** – you summarized data, visualized distributions, ran statistical tests, and reported what happened. Predictive modeling is fundamentally different: you are building a mathematical function that takes new, never-before-seen inputs and outputs a prediction about the future.

This distinction matters because a model that merely memorizes its training data will look impressive during development and fail in production. The technical term for this failure is **overfitting**: the model has learned the noise and quirks of the training data rather than the underlying patterns that generalize to new observations.

The opposite problem is **underfitting**: the model is too simple to capture the real patterns in the data. An underfitting model performs poorly on both training data and new data.

Generalization is the goal of every predictive model – the ability to perform well on data the model has never seen during training. Every technique in this module exists to measure, improve, or protect generalization.

Why does this matter practically? Imagine you build a model that predicts customer churn with 95% accuracy on your development data. Management deploys it. Within a month, it is wrong on half the cases. The model memorized which specific customers churned in the historical data rather than learning the general patterns (high support calls, short tenure, month-to-month contracts) that predict churn. The entire ML workflow you will learn this week is designed to prevent this failure.

---

2. The ML Workflow

Every supervised machine learning project follows the same high-level pipeline:

1. **Data collection and preparation** – Load data, handle missing values, encode categorical variables, scale numeric features.
2. **Data splitting** – Divide the data into training, validation, and test sets before any modeling.
3. **Preprocessing** – Apply transformations (scaling, encoding) fitted only on training data.
4. **Model training** – Fit a model on the training data.
5. **Evaluation** – Measure performance on held-out data using appropriate metrics.
6. **Iteration** – Adjust the model, hyperparameters, or features and re-evaluate.

The critical principle: **the test set is never used during training or model selection.** You split first, then do everything else. Violating this principle – even accidentally – produces artificially inflated performance estimates. This contamination is called **data leakage**, and it is one of the most common and damaging mistakes in applied ML.

This workflow is the same regardless of model type. In Week A you will use linear and logistic regression. In Week B you will drop in decision trees and random forests using the identical workflow. The scikit-learn **API** (Application Programming Interface) is designed around this principle – every model follows the same interface.

---

3. Data Splitting: Train, Validation, and Test Sets

Why Split?

If you train a model on all your data and then evaluate it on the same data, you are measuring memorization, not generalization. Splitting the data into separate subsets gives you an honest estimate of real-world performance.

The Three Splits

- **Training set** – The data the model learns from. The model sees these examples during `fit()`.
- **Validation set** – Used during development to compare model configurations, tune hyperparameters, and make modeling decisions. The model never trains on this data, but you look at validation performance repeatedly.
- **Test set** – Touched exactly once, at the very end, to get a final unbiased performance estimate. The test set is **sacred** – if you use it to make modeling decisions, it becomes a de facto validation set and your final estimate is no longer trustworthy.

Common Split Ratios

Strategy	Train	Validation	Test	When to Use
Simple holdout	80%	–	20%	Quick experiments, large data sets
Three-way split	70%	15%	15%	Hyperparameter tuning with explicit validation set
Three-way split	80%	10%	10%	Large data sets where 10% is still substantial

When you use cross-validation (Section 4), you often do not need a separate validation set – cross-validation handles model selection internally, and the test set is reserved for the final evaluation.

Splitting with scikit-learn

The `train_test_split` function from `sklearn.model_selection` handles splitting:

```python
from sklearn.model_selection import train_test_split

X = df.drop(columns=["churned"])
y = df["churned"]

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)
```

Key parameters:

- `test_size=0.2` – 20% of the data goes to the test set.
- `random_state=42` – Fixes the random seed so the split is reproducible. Without this, every run produces a different split, making results impossible to compare or debug. The specific number does not matter – consistency does.
- `stratify=y` – Ensures the class distribution in the target variable is preserved in both splits. If 26% of your data has `churned=1`, both the training set and test set will have approximately 26% churned. This is essential for classification tasks, especially with imbalanced classes.

Verifying Your Split

Always check that the split preserved the expected proportions:

```python
print(f"Training set size: {len(X_train)}")
print(f"Test set size:     {len(X_test)}")
print(f"Train churn rate:  {y_train.mean():.3f}")
print(f"Test churn rate:   {y_test.mean():.3f}")
```

If the class ratios diverge significantly between train and test, you likely forgot to set `stratify`.

---

4. Cross-Validation

Why a Single Split Is Not Enough

A single train/test split is sensitive to which specific rows end up in which set. You might get lucky or unlucky. **Cross-validation (CV)** produces a more reliable performance estimate by training and evaluating the model multiple times on different subsets of the data.

k-Fold Cross-Validation

In **k-fold CV**, the training data is divided into *k* equally sized subsets called **folds**. The model is trained *k* times, each time using *k-1* folds for training and 1 fold for validation. Every data point serves as a validation example exactly once.

Common choices for *k*: 5 or 10. Five-fold CV is a good default – it balances computational cost and estimate reliability.

Stratified k-Fold

For classification tasks, use **stratified k-fold CV** (`StratifiedKFold`). Standard k-fold does not guarantee that each fold has the same class distribution as the full dataset. Stratified k-fold preserves the class proportions in every fold, which is critical when classes are imbalanced.

Using `cross_val_score`

```python
from sklearn.model_selection import cross_val_score
from sklearn.linear_model import LogisticRegression

model = LogisticRegression(random_state=42, max_iter=1000)
scores = cross_val_score(model, X_train, y_train, cv=5, scoring="accuracy")

print(f"CV Accuracy: {scores.mean():.3f} +/- {scores.std():.3f}")
```

What this does:

1. Splits `X_train` into 5 stratified folds (stratified is the default for classifiers).
2. Trains the model 5 times, each time on 4 folds, evaluating on the held-out fold.
3. Returns an array of 5 scores.

Interpreting the result: Report as mean +/- standard deviation. A result of `0.79 +/- 0.03` means the model's accuracy is consistently around 79%, with low variance across folds. A high standard deviation (e.g., `0.79 +/- 0.12`) suggests the model's performance is unstable – it may be sensitive to which data it sees.

Using `cross_validate` for Multiple Metrics

When you need more than one metric, use `cross_validate` :

```python
from sklearn.model_selection import cross_validate

cv_results = cross_validate(
    model, X_train, y_train, cv=5,
    scoring=["accuracy", "precision", "recall", "f1"]
)

for metric in ["test_accuracy", "test_precision", "test_recall", "test_f1"]:
    scores = cv_results[metric]
    print(f"{metric}: {scores.mean():.3f} +/- {scores.std():.3f}")
```

When to Use CV vs. Holdout

- **Use cross-validation** when your dataset is small to moderate (under ~50,000 rows) and you need a reliable estimate of performance.
- **Use a simple holdout** when your dataset is very large (hundreds of thousands of rows), computational cost matters, and a single split provides a stable estimate.
- **Always reserve a final test set** regardless of whether you use CV for model selection.

TimeSeriesSplit

When your data has a temporal ordering (e.g., transactions over time), standard k-fold CV violates the assumption that future data should not be used to predict the past. `TimeSeriesSplit` from scikit-learn ensures that training folds always precede validation folds in time. This is not required for Week A assignments but is important to be aware of for time-ordered data sets.

---

5. Evaluation Metrics for Classification

Accuracy and Its Limitations

Accuracy is the fraction of predictions that are correct:

```
Accuracy = (correct predictions) / (total predictions)
```

Accuracy is intuitive but **misleading on imbalanced data sets**. If 95% of transactions are legitimate, a model that always predicts "legitimate" achieves 95% accuracy while catching zero fraud. Accuracy rewards ignoring the minority class.

The Confusion Matrix

The **confusion matrix** breaks predictions into four categories:

	Predicted Positive	Predicted Negative
Actually Positive	True Positive (TP)	False Negative (FN)
Actually Negative	False Positive (FP)	True Negative (TN)

- **True Positive (TP):** Model correctly predicted positive (e.g., correctly identified a churning customer).
- **False Positive (FP):** Model incorrectly predicted positive (e.g., flagged a loyal customer as churning).
- **True Negative (TN):** Model correctly predicted negative.
- **False Negative (FN):** Model incorrectly predicted negative (e.g., missed a churning customer).

Precision

Precision answers: "When the model says positive, how often is it right?"

```
Precision = TP / (TP + FP)
```

High precision means few false alarms. Precision matters when the cost of a false positive is high – for example, a spam filter that marks legitimate emails as spam (false positive) causes users to miss important messages.

Recall

Recall (also called sensitivity or true positive rate) answers: "Of all actual positives, how many did the model find?"

```
Recall = TP / (TP + FN)
```

High recall means few missed positives. Recall matters when the cost of missing a positive is high – for example, a disease screening test that misses an actual case (false negative) could be life-threatening.

F1-Score

The **F1-score** is the harmonic mean of precision and recall:

```
F1 = 2 * (Precision * Recall) / (Precision + Recall)
```

F1 balances precision and recall into a single number. It is useful when you need a single metric and care about both false positives and false negatives. The harmonic mean penalizes extreme imbalances – a model with 100% precision but 10% recall gets an F1 of 0.18, not 0.55.

> ***A note on F1 vs. PR-AUC.*** *F1 is a useful single-number summary for now, and it is what we will use throughout Module 5 Week A. In Module 5 Week B we will look at* ***PR-AUC*** *(the area under the precision-recall curve), which is the metric most practitioners reach for when working with imbalanced classification in practice. F1 measures the model at a single decision threshold; PR-AUC summarizes performance across all thresholds. Treat F1 as your Week A workhorse and expect Week B to add a more sophisticated view.*

When Each Metric Matters

Scenario	Priority Metric	Why
Spam filter	Precision	False positives (real email marked as spam) annoy users
Disease screening	Recall	False negatives (missed disease) are dangerous
Customer churn	Recall (often)	Missing a churning customer means lost revenue
Content moderation	Depends on policy	False positives (censoring valid content) vs. false negatives (missing harmful content)

Computing Metrics in scikit-learn

```python
from sklearn.metrics import (
    accuracy_score, precision_score, recall_score, f1_score,
    classification_report, confusion_matrix, ConfusionMatrixDisplay
)

y_pred = model.predict(X_test)

print(f"Accuracy:  {accuracy_score(y_test, y_pred):.3f}")
print(f"Precision: {precision_score(y_test, y_pred):.3f}")
print(f"Recall:    {recall_score(y_test, y_pred):.3f}")
print(f"F1:        {f1_score(y_test, y_pred):.3f}")

# Full classification report
print(classification_report(y_test, y_pred))
```

Visualizing the Confusion Matrix

```python
import matplotlib.pyplot as plt

cm = confusion_matrix(y_test, y_pred)
disp = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=["No Churn", "C
disp.plot(cmap="Blues")
plt.title("Confusion Matrix — Logistic Regression")
plt.show()
```

`ConfusionMatrixDisplay` provides a clear heatmap visualization. Always label the axes with meaningful class names rather than 0/1.

---

6. Evaluation Metrics for Regression

Regression models predict continuous values. Different metrics capture different aspects of prediction quality.

Mean Squared Error (MSE)

```
MSE = (1/n) * sum((y_true - y_pred)^2)
```

MSE squares each error before averaging. This means large errors are penalized much more heavily than small errors. MSE is useful when large errors are particularly undesirable.

Root Mean Squared Error (RMSE)

```
RMSE = sqrt(MSE)
```

RMSE is MSE returned to the original units of the target variable. If you are predicting monthly charges in dollars, RMSE is in dollars. This makes it easier to interpret: an RMSE of $10 means "on average, predictions are about $10 off."

Mean Absolute Error (MAE)

```
MAE = (1/n) * sum(|y_true - y_pred|)
```

MAE averages the absolute errors without squaring. Unlike MSE/RMSE, MAE treats all errors equally – a $20 error counts twice as much as a $10 error, not four times as much. MAE is more robust to outliers than RMSE.

When to prefer MAE over RMSE: Use MAE when you want a metric that is not disproportionately influenced by a few extreme predictions. Use RMSE when large errors are genuinely worse than small errors in your application (e.g., predicting building structural loads, where being off by a lot is much worse than being off by a little).

R-squared (Coefficient of Determination)

```
R^2 = 1 - (sum((y_true - y_pred)^2) / sum((y_true - y_mean)^2))
```

R-squared measures how much better the model is than always predicting the mean of the target variable.

- **R-squared = 1:** Perfect predictions (every prediction equals the true value).
- **R-squared = 0:** The model is no better than predicting the mean.
- **R-squared < 0:** The model is worse than predicting the mean (a sign of a serious problem).

R-squared is a relative metric – it tells you how much of the variance in the target your model explains.

Computing Regression Metrics in scikit-learn

```python
from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score
import numpy as np

y_pred = model.predict(X_test)

mse  = mean_squared_error(y_test, y_pred)
rmse = np.sqrt(mse)
mae  = mean_absolute_error(y_test, y_pred)
r2   = r2_score(y_test, y_pred)

print(f"MSE:  {mse:.2f}")
print(f"RMSE: {rmse:.2f}")
print(f"MAE:  {mae:.2f}")
print(f"R^2:  {r2:.3f}")
```

---

7. Linear Regression

What Linear Regression Does

Linear regression models the relationship between input features and a continuous target as a weighted sum:

```
y = w1*x1 + w2*x2 + ... + wn*xn + b
```

Each feature *x_i* gets a coefficient (weight) *w_i*, and *b* is the intercept (bias term). The model finds the weights that minimize the sum of squared errors between predictions and actual values – this is called **Ordinary Least Squares (OLS)**.

Assumptions

Linear regression assumes:

1. **Linearity** – The relationship between features and target is approximately linear.
2. **Independence** – Observations are independent of each other.
3. **Homoscedasticity** – The variance of errors is constant across all levels of the features (errors do not fan out or narrow as predictions change).

These assumptions are rarely perfectly met in practice. Linear regression often still works reasonably well when they are approximately satisfied. When they are severely violated, the model's predictions and coefficient interpretations become unreliable.

Interpreting Coefficients

Each coefficient tells you: "Holding all other features constant, a one-unit increase in this feature is associated with a *w_i*-unit change in the target." For example, a coefficient of 12.5 on `tenure` means that each additional month of tenure is associated with $12.50 higher monthly charges, all else being equal.

Using LinearRegression in scikit-learn

```python
from sklearn.linear_model import LinearRegression

model = LinearRegression()
model.fit(X_train, y_train)

y_pred = model.predict(X_test)

print(f"R^2: {model.score(X_test, y_test):.3f}")
print(f"Coefficients: {model.coef_}")
print(f"Intercept: {model.intercept_:.3f}")
```

Limitations

- Cannot capture nonlinear relationships without manual feature engineering (e.g., adding polynomial terms).
- Sensitive to **multicollinearity** – when features are highly correlated with each other, individual coefficient estimates become unstable (they can swing wildly between runs on slightly different data). The model's predictions may still be reasonable, but interpreting individual coefficients becomes unreliable. Regularization (Section 9) directly addresses this problem.
- No built-in way to model interactions between features.

---

8. Logistic Regression

From Regression to Classification

Despite its name, **logistic regression** is a classification algorithm. It models the probability that an observation belongs to the positive class. The key idea: instead of predicting a continuous value, it predicts a probability between 0 and 1, then applies a threshold (default: 0.5) to make a binary decision.

The Sigmoid Function

Logistic regression applies the **sigmoid function** to a linear combination of features:

```
P(y=1) = 1 / (1 + e^(-(w1*x1 + w2*x2 + ... + b)))
```

The sigmoid transforms any real number into a value between 0 and 1. Values above 0.5 are classified as positive (class 1); values below 0.5 are classified as negative (class 0). The **decision boundary** is the surface in feature space where the predicted probability equals 0.5.

Interpreting Coefficients as Log-Odds

In logistic regression, coefficients represent **log-odds**. A coefficient of 0.8 on `num_support_calls` means that each additional support call multiplies the odds of churning by e^0.8 (approximately 2.23). Positive coefficients increase the probability of the positive class; negative coefficients decrease it.

Using LogisticRegression in scikit-learn

```python
from sklearn.linear_model import LogisticRegression

model = LogisticRegression(random_state=42, max_iter=1000)
model.fit(X_train, y_train)

y_pred = model.predict(X_test)
y_proba = model.predict_proba(X_test)[:, 1]  # Probability of positive class

print(f"Accuracy: {model.score(X_test, y_test):.3f}")
```

Key parameters:

- `max_iter=1000` – Maximum iterations for the solver to converge. The default (100) is often insufficient for larger or scaled data sets. If you see a `ConvergenceWarning`, increase `max_iter`.
- `random_state=42` – For reproducibility.
- `solver` – The optimization algorithm. `lbfgs` (the default) works well for L2 regularization. For L1 regularization or ElasticNet, use `solver='saga'`. More on this in Section 9.

Probabilities vs. Predictions

`predict()` returns hard class labels (0 or 1). `predict_proba()` returns the estimated probability for each class. The probabilities are often more useful than the hard predictions – they tell you how confident the model is. A customer with a 0.95 churn probability is a much higher priority for intervention than one with 0.52, even though both are classified as "will churn."

---

9. Regularization

Why Regularization Exists

When a model has many features – especially correlated features – it can overfit by assigning large, volatile coefficients that capture noise rather than signal. **Regularization** adds a penalty to the model's loss function that constrains the size of the coefficients, forcing the model to find simpler solutions that generalize better.

Regularization also helps with **multicollinearity**: when features are highly correlated, unregularized models produce unstable coefficients that change dramatically with small data perturbations. Regularization stabilizes these estimates.

L2 Regularization (Ridge)

L2 regularization (also called **Ridge**) adds the sum of squared coefficients to the loss function:

```
Loss = Original Loss + alpha * sum(w_i^2)
```

Effect: shrinks all coefficients toward zero but never exactly to zero. Every feature stays in the model, but extreme coefficients are pulled back. The `alpha` parameter controls the strength of the penalty – larger alpha means more shrinkage.

```python
from sklearn.linear_model import Ridge

model = Ridge(alpha=1.0)
model.fit(X_train, y_train)
```

L1 Regularization (Lasso)

L1 regularization (also called **Lasso**) adds the sum of absolute coefficient values:

```
Loss = Original Loss + alpha * sum(|w_i|)
```

Effect: can drive coefficients to exactly zero, effectively performing **feature selection**. Features with zero coefficients are eliminated from the model. This is useful when you suspect many features are irrelevant.

```python
from sklearn.linear_model import Lasso

model = Lasso(alpha=0.1)
model.fit(X_train, y_train)

# See which features were eliminated
for name, coef in zip(feature_names, model.coef_):
    print(f"{name}: {coef:.4f}")
```

ElasticNet

ElasticNet combines L1 and L2 penalties:

```
Loss = Original Loss + alpha * (l1_ratio * sum(|w_i|) + (1 - l1_ratio) * sum(w_i^
```

The `l1_ratio` parameter controls the mix: `l1_ratio=1` is pure Lasso, `l1_ratio=0` is pure Ridge. ElasticNet is useful when you have groups of correlated features – Lasso tends to arbitrarily pick one from each group, while ElasticNet keeps the group together.

Regularization in LogisticRegression

In scikit-learn's `LogisticRegression`, regularization is controlled by the `C` parameter, which is the **inverse** of regularization strength:

- **Large C** (e.g., 100) = weak regularization (the model is relatively unconstrained).
- **Small C** (e.g., 0.01) = strong regularization (the model is heavily constrained).

This inverse relationship is counterintuitive. Think of C as "confidence in the data" – large C means "trust the data more," small C means "trust the data less and keep the model simple."

```python
# Strong regularization with L1 penalty
model = LogisticRegression(C=0.1, penalty="l1", solver="saga", random_state=42, m
model.fit(X_train, y_train)
```

Solver selection for LogisticRegression:

Penalty	Compatible Solvers
L2 (default)	`lbfgs` (default), `saga`, `newton-cg`
L1	`saga`, `liblinear`
ElasticNet	`saga` only

If you switch the penalty to `l1` or `elasticnet` without changing the solver from the default `lbfgs`, scikit-learn will raise an error. Use `solver='saga'` as a safe choice that works with all penalty types.

Choosing Regularization Strength with Cross-Validation

The regularization parameter (`alpha` for Ridge/Lasso, `C` for LogisticRegression) is a **hyperparameter** – a setting you choose before training, not a value the model learns from data. Use cross-validation to find the best value:

```python
from sklearn.model_selection import cross_val_score
from sklearn.linear_model import LogisticRegression

for c_val in [0.01, 0.1, 1.0, 10.0, 100.0]:
    model = LogisticRegression(C=c_val, random_state=42, max_iter=1000)
    scores = cross_val_score(model, X_train, y_train, cv=5, scoring="f1")
    print(f"C={c_val:6.2f}  F1: {scores.mean():.3f} +/- {scores.std():.3f}")
```

The value that produces the best CV score is your choice. If multiple values produce similar scores, prefer the simpler model (stronger regularization).

---

10. The Bias-Variance Tradeoff

Two Sources of Error

Every model's prediction error comes from two sources:

- **Bias** – Error from overly simplistic assumptions. A model with high bias **underfits**: it cannot capture the true patterns in the data. Example: fitting a straight line to data that follows a curve.
- **Variance** – Error from sensitivity to the specific training data. A model with high variance **overfits**: it fits the training data perfectly but performs poorly on new data because it has memorized noise.

Model Complexity as a Dial

Think of model complexity as a dial that trades bias for variance:

- **Turn the dial toward simple** (e.g., strong regularization, fewer features): bias increases, variance decreases. The model is stable but may miss real patterns.
- **Turn the dial toward complex** (e.g., weak regularization, many features, polynomial terms): bias decreases, variance increases. The model captures more patterns but risks memorizing noise.

The sweet spot is the complexity level that minimizes **total error** (bias + variance). This is the bias-variance tradeoff.

How Regularization Controls the Tradeoff

Regularization is a direct mechanism for navigating this tradeoff:

- **No regularization:** The model is free to fit the training data as closely as possible. Low bias, potentially high variance.
- **Moderate regularization:** The model is constrained but still flexible enough to capture real patterns. Balanced bias and variance.
- **Heavy regularization:** The model is heavily constrained. High bias (underfitting), low variance.

Learning Curves as a Diagnostic

A **learning curve** plots model performance as a function of training set size. It reveals whether your model suffers from high bias or high variance:

- **High bias (underfitting):** Both training and validation scores are low and converge early. More data will not help – the model is too simple.
- **High variance (overfitting):** Training score is high, validation score is much lower, and there is a large gap. More data may help by giving the model more examples to generalize from.

You will work with learning curves in the Week A stretch assignment. For now, understand the concept: learning curves are a diagnostic tool for the bias-variance tradeoff.

---

11. scikit-learn API Fundamentals

An **API** – Application Programming Interface – is the set of classes, methods, and conventions that a library exposes for you to use. When we say "scikit-learn's API," we mean the specific methods (`fit`, `predict`, `transform`), the parameter conventions, and the input/output contracts that every scikit-learn object follows. A consistent API means you learn one pattern and it works for any algorithm – swap `LogisticRegression` for `RandomForestClassifier` and the calling code stays the same.

The Estimator Pattern

Every scikit-learn model is an **estimator** – an object with three core methods:

- `fit(X, y)` – Learn from training data. For a classifier, this means finding the optimal coefficients. For a scaler, this means computing the mean and standard deviation.
- `predict(X)` – Generate predictions for new data.
- `score(X, y)` – Compute a default performance metric (accuracy for classifiers, R-squared for regressors).

```python
from sklearn.linear_model import LogisticRegression

model = LogisticRegression(random_state=42, max_iter=1000)
model.fit(X_train, y_train)        # Learn
predictions = model.predict(X_test)  # Predict
accuracy = model.score(X_test, y_test)  # Evaluate
```

A **hyperparameter** is any parameter set before `fit()` is called – the model does not learn it from data. Examples: `C` in LogisticRegression, `alpha` in Ridge, `max_iter`. Hyperparameters are passed to the constructor.

Transformer Objects

A **transformer** is an estimator that also has a `transform(X)` method. These objects modify data rather than making predictions. The most common example is `StandardScaler`, which scales features to have zero mean and unit variance:

```python
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()
scaler.fit(X_train)                     # Compute mean and std from training data
X_train_scaled = scaler.transform(X_train)  # Apply to training data
X_test_scaled = scaler.transform(X_test)    # Apply to test data using TRAINING s
```

The `fit_transform(X)` shortcut combines `fit()` and `transform()` in one call:

```python
X_train_scaled = scaler.fit_transform(X_train)  # Fit and transform in one step
X_test_scaled = scaler.transform(X_test)          # Transform only -- do NOT refit
```

Critical rule: The scaler (or any transformer) must be fit **only on training data**. If you fit on the full dataset before splitting, the mean and standard deviation include information from the test set – this is data leakage. The test set's statistics contaminate the training process, producing overly optimistic performance estimates.

Pipelines

A **pipeline** chains preprocessing steps and a model into a single object. This is the recommended way to build ML workflows in scikit-learn because it prevents data leakage automatically.

```python
from sklearn.pipeline import Pipeline, make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression

# Explicit Pipeline construction
pipe = Pipeline([
    ("scaler", StandardScaler()),
    ("classifier", LogisticRegression(random_state=42, max_iter=1000))
])

# Shorthand with make_pipeline (auto-generates step names)
pipe = make_pipeline(StandardScaler(), LogisticRegression(random_state=42, max_it

# Use exactly like a single model
pipe.fit(X_train, y_train)
predictions = pipe.predict(X_test)
accuracy = pipe.score(X_test, y_test)
```

Why Pipelines Prevent Data Leakage

When you call `pipe.fit(X_train, y_train)` :

1. The `StandardScaler` is fit on `X_train` (computes mean and std).
2. `X_train` is transformed using those statistics.
3. The `LogisticRegression` is fit on the scaled training data.

When you call `pipe.predict(X_test)` :

1. `X_test` is transformed using the **training** statistics (the scaler was already fit).
2. The model makes predictions on the scaled test data.

The scaler never sees the test data during fitting. This guarantee holds even during cross-validation – each fold gets its own scaler fit only on that fold's training data. Without a Pipeline, you would need to manually manage this for every fold.

Pipelines with ColumnTransformer

Real data sets have mixed types – numeric features need scaling, categorical features need encoding. `ColumnTransformer` handles this by applying different transformations to different column groups:

```python
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.pipeline import Pipeline
from sklearn.linear_model import LogisticRegression

numeric_features = ["tenure", "monthly_charges", "total_charges", "num_support_ca
categorical_features = ["contract_type", "internet_service", "payment_method"]

preprocessor = ColumnTransformer(
    [
        ("num", StandardScaler(), numeric_features),
        ("cat", OneHotEncoder(drop="first", handle_unknown="ignore"), categorical
    ]
)

pipe = Pipeline([
    ("preprocessor", preprocessor),
    ("classifier", LogisticRegression(random_state=42, max_iter=1000))
])

pipe.fit(X_train, y_train)
print(f"Accuracy: {pipe.score(X_test, y_test):.3f}")
```

The `ColumnTransformer` + `Pipeline` pattern is the standard way to handle mixed-type data in scikit-learn. You will use this pattern extensively in the integration assignment.

Key Terms

- **Generalization** – A model's ability to perform well on data it was not trained on.
- **Overfitting** – The model has learned noise in the training data and performs well on training data but poorly on new data.
- **Underfitting** – The model is too simple to capture the real patterns; it performs poorly on both training and new data.
- **Training set** – The data used to fit the model.
- **Validation set** – Data used during development to compare model configurations and tune hyperparameters; the model is not trained on it.
- **Test set** – Data used once at the end to estimate final model performance; never used for model selection.
- **Holdout** – The strategy of setting aside a portion of data for evaluation.
- **Cross-validation** – A technique that trains and evaluates the model multiple times on different subsets, producing a more reliable performance estimate.
- **k-fold** – A cross-validation strategy that divides data into k subsets, using each as a validation set once.
- **Stratified k-fold** – k-fold CV that preserves class proportions in each fold.
- **Fold** – One of the k subsets in k-fold cross-validation.
- **Accuracy** – Fraction of predictions that are correct.
- **Precision** – Of all positive predictions, the fraction that are actually positive (TP / (TP + FP)).
- **Recall** – Of all actual positives, the fraction the model found (TP / (TP + FN)).
- **F1-score** – Harmonic mean of precision and recall.
- **Confusion matrix** – A table showing TP, FP, TN, FN counts for a classifier.
- **True positive (TP)** – Model correctly predicts positive.
- **False positive (FP)** – Model incorrectly predicts positive.
- **True negative (TN)** – Model correctly predicts negative.
- **False negative (FN)** – Model incorrectly predicts negative.
- **MSE (Mean Squared Error)** – Average of squared differences between predictions and true values.
- **RMSE (Root Mean Squared Error)** – Square root of MSE; in the same units as the target variable.
- **MAE (Mean Absolute Error)** – Average of absolute differences between predictions and true values.
- **R-squared (Coefficient of determination)** – Proportion of variance in the target explained by the model; 1 is perfect, 0 is no better than the mean.
- **Linear regression** – A model that predicts a continuous target as a weighted sum of features.
- **Logistic regression** – A classification model that predicts the probability of the positive class using the sigmoid function.
- **Decision boundary** – The surface in feature space where the predicted probability equals the classification threshold (typically 0.5).
- **Regularization** – A penalty added to the loss function to constrain coefficient magnitude and reduce overfitting.
- **L1 (Lasso)** – Regularization that adds the sum of absolute coefficient values; can drive coefficients to zero (feature selection).
- **L2 (Ridge)** – Regularization that adds the sum of squared coefficient values; shrinks coefficients toward zero but not exactly to zero.
- **ElasticNet** – Regularization that combines L1 and L2 penalties.
- **Bias-variance tradeoff** – The tension between a model's ability to fit data (low bias) and its stability across different data sets (low variance).
- **Model complexity** – The capacity of a model to fit intricate patterns; controlled by features, parameters, and regularization.
- **Estimator** – Any scikit-learn object with a `fit()` method.
- **Transformer** – An estimator that also has `transform()` and `fit_transform()` methods; modifies data rather than making predictions.
- **Pipeline** – A scikit-learn object that chains preprocessing steps and a final estimator, ensuring correct fitting order and preventing data leakage.
- **Data leakage** – When information from outside the training set (e.g., test set statistics) contaminates the model training process, producing artificially high performance estimates.
- **Hyperparameter** – A model setting chosen before training (e.g., C, alpha, max_iter); not learned from data.
- **random_state** – A parameter that fixes the random seed for reproducibility.
- **stratify** – A parameter in `train_test_split` that preserves class proportions in the splits.

Summary

This reading covered the complete foundation for predictive modeling with scikit-learn:

1. **From description to prediction** – Generalization is the goal; overfitting and underfitting are the failure modes.
2. **The ML workflow** – Split first, then preprocess, train, evaluate, iterate. Never touch the test set during development.
3. **Data splitting** – Train/validation/test serve different purposes. Use `stratify` for classification. Set `random_state` for reproducibility.
4. **Cross-validation** – k-fold and stratified k-fold produce more reliable performance estimates than a single split.
5. **Classification metrics** – Accuracy alone is insufficient. Precision, recall, F1, and the confusion matrix tell a complete story.
6. **Regression metrics** – MSE, RMSE, MAE, and R-squared each capture different aspects of prediction quality.
7. **Linear regression** – OLS fitting, coefficient interpretation, and the linearity assumption.
8. **Logistic regression** – Probability estimation for binary classification via the sigmoid function.
9. **Regularization** – L1 (Lasso) for feature selection, L2 (Ridge) for coefficient shrinkage, ElasticNet for both. C is inverse regularization strength.
10. **Bias-variance tradeoff** – Underfitting vs. overfitting, with regularization as the control mechanism.
11. **scikit-learn API** – Estimators (`fit` / `predict` / `score`), preprocessing steps (`fit_transform`), and Pipelines for leak-proof workflows.

What's Next

In **Week B**, you will build on this foundation by adding more powerful model types – decision trees and random forests – to your toolkit. You will learn to handle class imbalance using class weights, threshold tuning, and resampling. You will interpret calibrated probabilities and evaluate models with PR-AUC and calibration curves. The Week B integration task extends the evaluation pipeline you build this week into a full model comparison with a decision memo – the culminating deliverable for Module 5.

© 2026 LevelUp Economy. All rights reserved. Unauthorized reproduction or distribution of this material is prohibited.